Smart IoT Agriculture System Using NodeMCU ESP8266

Abstract:

The Smart IoT Agriculture System is designed to optimize irrigation in hilly regions, particularly in areas like Jorethang, South Sikkim, which face water scarcity during hot summers. The system integrates soil moisture, temperature, humidity, and water tank level sensors with a NodeMCU ESP8266 microcontroller, enabling automated and manual control of a water pump through both physical switches and a mobile app (Blynk IoT). The system provides real-time monitoring on an LCD and mobile app, and ensures water-efficient irrigation by preventing pump operation when the tank level is low. This solution minimizes water wastage, enhances crop productivity, and provides remote management and notifications to farmers, contributing to sustainable agriculture.

The system offers automatic irrigation based on soil moisture and water availability and allows manual control via a mobile app (Blynk) or a physical switch. Real-time monitoring is displayed on LCD and a mobile dashboard, helping farmers save water, improve crop yield, and reduce labor.

Additionally, the system prevents water pump operation when the water tank is low, ensuring sustainable water management in hilly agriculture.

Introduction:

Agriculture in hilly regions faces unique challenges, including uneven terrain, soil erosion, and water scarcity. Traditional irrigation methods are inefficient and require significant manual labor. With increasing climate variability and prolonged dry seasons, smart, sensor-driven irrigation systems are crucial to enhance water use efficiency.

This project implements a Smart IoT Agriculture System using NodeMCU ESP8266, which:

- Monitors soil moisture, temperature, and humidity.
- Monitors rainwater harvesting tank levels.
- Controls water pumps automatically based on soil conditions and water availability.
- Allows manual control via physical switches or a mobile app (Blynk).
- Provides real-time data visualization on LCD and mobile app.

The goal is to conserve water, improve crop yield, and reduce manual labor in regions like Jorethang, South Sikkim.

Why IoT-based Smart Irrigation?

- Automated Monitoring: Sensors track soil moisture, temperature, and humidity.
- Efficient Water Use: Pumps operate only when needed.
- Remote Control: Farmers can manage irrigation via mobile apps.
- Integration with Rainwater Harvesting: Ensures water availability even in dry periods.

This system addresses the need for precision irrigation in hilly terrains, optimizing water usage and increasing crop productivity.

Literature Review / Background of the Work:

Smart irrigation systems have been widely studied and implemented to address water scarcity and improve agricultural efficiency. For instance, integrating rainwater harvesting with smart irrigation has been shown to conserve water and reduce costs. The use of NodeMCU ESP8266 in agricultural IoT applications allows for real-time monitoring and control, enhancing decision-making and resource management. Additionally, combining soil moisture sensors with cloud platforms like Blynk enables remote monitoring and data analysis, further optimizing irrigation processes.

- Challenges in Hilly Agriculture: Steep terrain and poor water availability make traditional irrigation methods ineffective.
- Smart Irrigation: IoT-based systems have been proven to improve water efficiency by monitoring soil and environmental parameters.
- NodeMCU ESP8266: A popular microcontroller with WiFi capabilities for remote monitoring.
- Rainwater Harvesting: Storing rainwater ensures continuous irrigation during dry seasons.
    - Sensor-based irrigation using Arduino for flat regions, lacked remote monitoring.
    - IoT-based water management for crops in plains.
    - Limited work exists for hilly terrains, which require careful water management and remote monitoring.
- Rainwater Harvesting Systems: Stored water but not integrated with automated irrigation.

Traditional Irrigation Systems:

- Manual irrigation depends on farmer availability.
- Water wastage occurs when crops are overwatered.
- Difficult to measure soil moisture accurately in hilly regions.

Methodology:

Introduction

The system uses sensors and NodeMCU to collect data and control a water pump. Data is displayed in real-time on an LCD and mobile app. Users can switch between manual and automatic modes, and the pump is prevented from operating when water is insufficient.

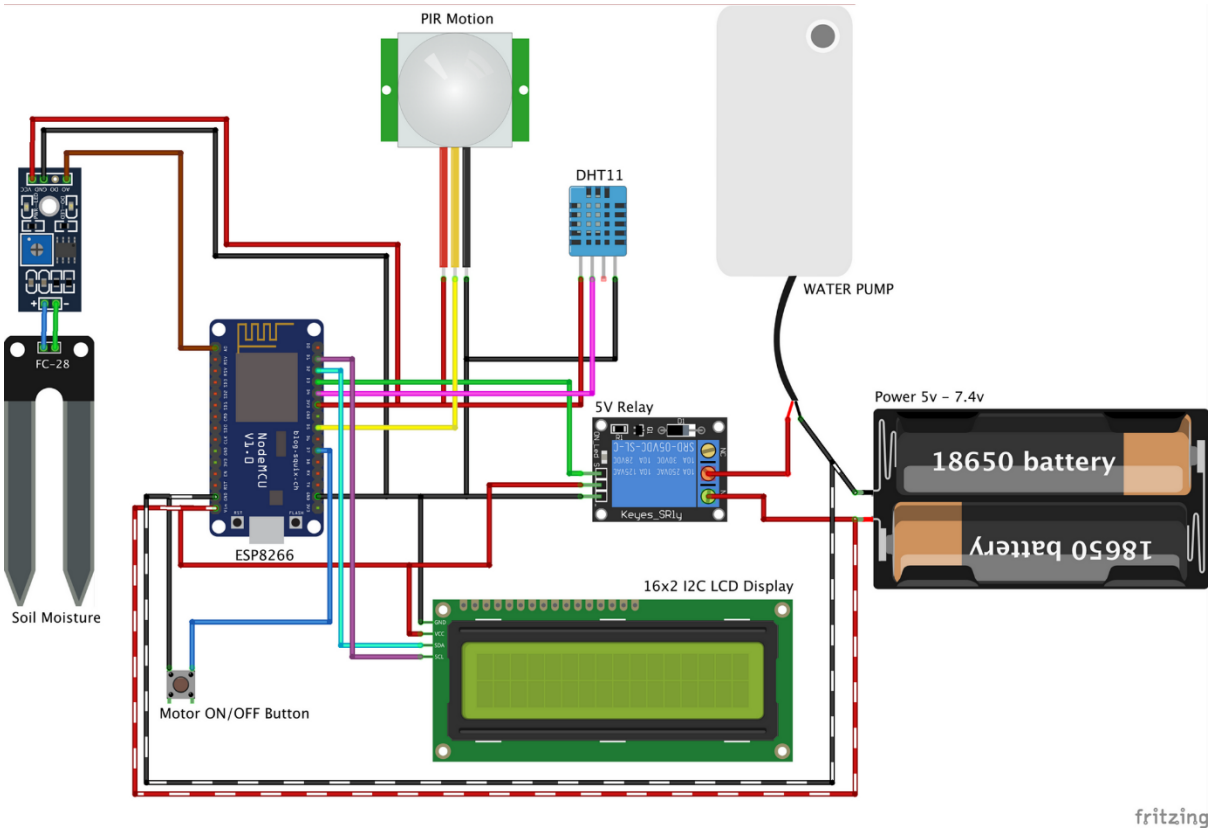
Components List:

- NodeMCU ESP8266
- DHT11 (Temperature & Humidity sensor)
- Soil Moisture Sensor
- Ultrasonic Sensor (HC-SR04) for water tank level
- Relay Module (to control water pump)
- LCD 16x2 with I2C
- Push Button (Manual Switch)
- Water Pump and power supply

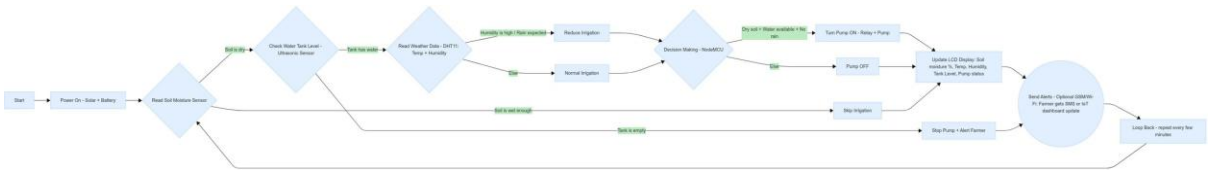
- NodeMCU ESP8266: Acts as the central controller, connecting sensors and actuators to the internet.
- Soil Moisture Sensor: Measures the volumetric water content in the soil.
- DHT11 Sensor: Monitors ambient temperature and humidity.
- Ultrasonic Sensor (HC-SR04): Assesses the water level in the rainwater harvesting tank.
- Relay Module: Controls the water pump based on sensor inputs.
- LCD Display: Displays real-time sensor data locally.

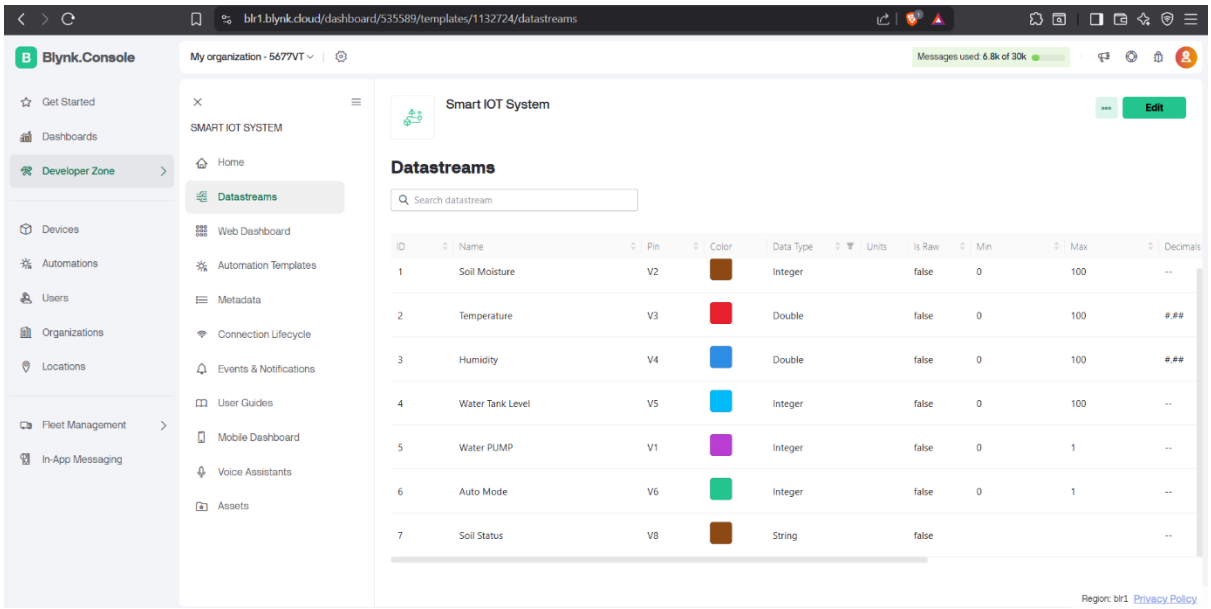
- Blynk App: Provides a user-friendly interface for remote monitoring and control.

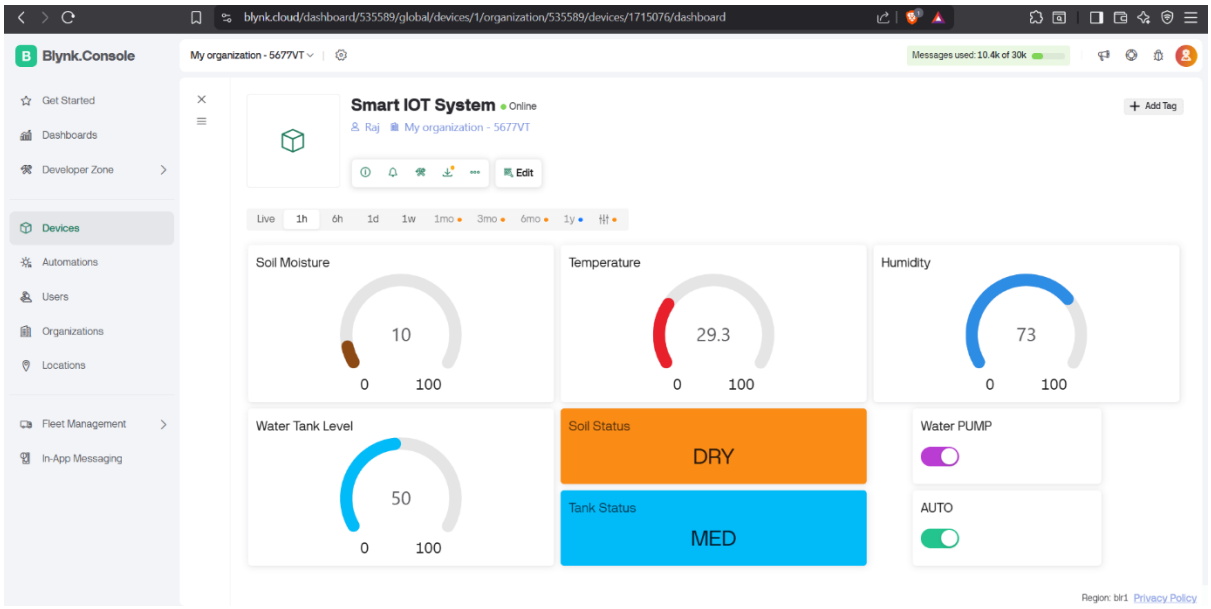
Circuit Diagram:

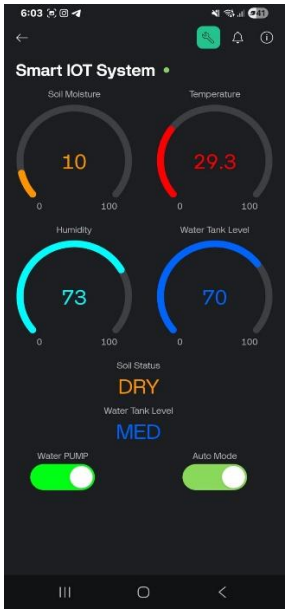


Flow Chart:




blr1.blynk.cloud/dashboard/535589/templates/1132724/datastreams
Blynk.Console
My organization - 5677VT
Messages used: 6.8k of 30k
Get Started
Dashboards
Developer Zone
Devices
Automations
Users
Organizations
Locations
Fleet Management
In-App Messaging
SMART IOT SYSTEM
Home
Datastreams
Web Dashboard
Automation Templates
Metadata
Connection Lifecycle
Events & Notifications
User Guides
Mobile Dashboard
Voice Assistants
Assets
Smart IOT System
Edit
Datastreams
Search datastream
ID Name Pin Color Data Type Units Is Raw Min Max Decimals
1 Soil Moisture V2 Integer false 0 100 --
2 Temperature V3 Double false 0 100 #.##
3 Humidity V4 Double false 0 100 #.##
4 Water Tank Level V5 Integer false 0 100 --
5 Water PUMP V1 Integer false 0 1 --
6 Auto Mode V6 Integer false 0 1 --
7 Soil Status V8 String false --
Region: blr1 Privacy Policy


blynk.cloud/dashboard/535589/global/devices/1/organization/535589/devices/1715076/dashboard
Blynk.Console
My organization - 5677VT
Messages used: 10.4k of 30k
Get Started
Dashboards
Developer Zone
Devices
Automations
Users
Organizations
Locations
Fleet Management
In-App Messaging
Smart IOT System
Online
Raj
My organization - 5677VT
Edit
Add Tag
Live 1h 6h 1d 1w 1mo 3mo 6mo 1y
Soil Moisture
10
0 100
Temperature
29.3
0 100
Humidity
73
0 100
Water Tank Level
50
0 100
Soil Status
DRY
Tank Status
MED
Water PUMP
AUTO
Region: blr1 Privacy Policy



Software / Algorithm:

1. Initialization: Sensors, LCD, Blynk connection, relay, and mode variables.
2. Loop Execution:
    - Read soil moisture, temperature, humidity, and water tank level.
    - Display data on LCD.
    - Send data to Blynk app.
    - Manual Control: Button toggles pump (only in manual mode).
    - Automatic Control:
        - If soil < threshold AND tank > minimum → pump ON
        - Else → pump OFF
3. Safety: Prevents pump from operating if water level is too low.
4. Blynk App Integration:
    - Soil moisture (%), soil status (DRY/WET)
    - Temperature, humidity
    - Water tank level (%), tank status (HIGH/MED/LOW)

- Pump status (ON/OFF)
- Auto/manual mode switch

Code:

```
// ====== BLYNK AUTHENTICATION ======

#define BLYNK_TEMPLATE_ID "TMPL3DhYBPkOS"

#define BLYNK_TEMPLATE_NAME "Smart IOT System"

#define BLYNK_AUTH_TOKEN "jppRBzNYhb3cO3NUDF2IOU1EKJNo_fRp"


// ====== LIBRARIES ======

#include <ESP8266WiFi.h>

#include <BlynkSimpleEsp8266.h>

#include <DHT.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>


// ====== WIFI CREDENTIALS ======

char auth[] = BLYNK_AUTH_TOKEN;

char ssid[] = "Raj";

char pass[] = "Rj@01010101";


// ====== PIN CONFIGURATION ======

#define DHTPIN D4

#define DHTTYPE DHT11

#define SOIL A0

#define RELAY D0
```

```
#define RELAY_ON  LOW

#define RELAY_OFF HIGH

#define TRIG D5

#define ECHO D6

#define MANUAL_SWITCH D7


// ====== OBJECTS ======

DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2);


// ====== CONTROL VARIABLES ======

bool autoMode = false;   // Auto (true) / Manual (false)

int pumpState = RELAY_OFF;


// ====== THRESHOLDS ======

int soilThreshold = 50;        // Soil % threshold

int waterTankHeight = 30;       // Tank height in cm


// ====== BLYNK FUNCTIONS ======

BLYNK_WRITE(V1) {  // Pump manual control

  if (!autoMode) {

    int pinValue = param.asInt();

    pumpState = (pinValue == 1) ? RELAY_ON : RELAY_OFF;
```

```
    digitalWrite(RELAY, pumpState);

    Blynk.virtualWrite(V7, (pumpState == RELAY_ON) ? 1 : 0);

  }

}


BLYNK_WRITE(V6) {  // Mode switch Auto/Manual

  int mode = param.asInt();

  autoMode = (mode == 1);  // 1 = Auto, 0 = Manual

}


void setup() {

  pinMode(RELAY, OUTPUT);

  digitalWrite(RELAY, RELAY_OFF);


  pinMode(TRIG, OUTPUT);

  pinMode(ECHO, INPUT);


  pinMode(MANUAL_SWITCH, INPUT_PULLUP);


  dht.begin();

  Blynk.begin(auth, ssid, pass);


  Wire.begin(D2, D1);  // LCD I2C pins
```

```
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("AgroSmart Ready");
  delay(2000);
  lcd.clear();
}

void loop() {
  Blynk.run();

  // ====== SENSOR READINGS ======
  int soilRaw = analogRead(SOIL);
  int soilPercent = map(soilRaw, 1023, 0, 0, 100);
  soilPercent = constrain(soilPercent, 0, 100);

  float temp = dht.readTemperature();
  float hum  = dht.readHumidity();

  // Ultrasonic water level
  long duration, distance;
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
```

```
digitalWrite(TRIG, LOW);

duration = pulseIn(ECHO, HIGH, 30000);

distance = (duration * 0.034) / 2;


int waterLevel = map(distance, waterTankHeight, 0, 0, 100);

waterLevel = constrain(waterLevel, 0, 100);


// ====== PHYSICAL SWITCH CONTROL ======

if (digitalRead(MANUAL_SWITCH) == LOW && !autoMode) {

  pumpState = (pumpState == RELAY_ON) ? RELAY_OFF : RELAY_ON; // toggle

  digitalWrite(RELAY, pumpState);

  Blynk.virtualWrite(V1, (pumpState == RELAY_ON) ? 1 : 0);

  Blynk.virtualWrite(V7, (pumpState == RELAY_ON) ? 1 : 0);

  delay(500); // debounce

}


// ====== AUTO MODE LOGIC ======

if (autoMode) {

  if (soilPercent < soilThreshold && waterLevel > 30) {

    pumpState = RELAY_ON;   // Soil DRY + Tank not LOW → Pump ON

  } else {

    pumpState = RELAY_OFF;  // Soil WET OR Tank LOW → Pump OFF

  }
```

```
  digitalWrite(RELAY, pumpState);

  Blynk.virtualWrite(V1, (pumpState == RELAY_ON) ? 1 : 0);

  Blynk.virtualWrite(V7, (pumpState == RELAY_ON) ? 1 : 0);

}


// ====== STATUS STRINGS ======

String soilStatus = (soilPercent < soilThreshold) ? "DRY" : "WET";


String waterStatus;

if (waterLevel > 70)      waterStatus = "HIGH";

else if (waterLevel > 30) waterStatus = "MED";

else                      waterStatus = "LOW";


String pumpStatus = (pumpState == RELAY_ON) ? "ON" : "OFF";


// ====== SEND TO BLYNK ======

Blynk.virtualWrite(V2, soilPercent);   // Soil %

Blynk.virtualWrite(V8, soilStatus);    // Soil DRY/WET

Blynk.virtualWrite(V3, temp);          // Temp

Blynk.virtualWrite(V4, (int)hum);      // Humidity (no decimal)

Blynk.virtualWrite(V5, waterLevel);    // Tank %

Blynk.virtualWrite(V9, waterStatus);   // Water HIGH/MED/LOW

Blynk.virtualWrite(V7, (pumpState == RELAY_ON) ? 1 : 0);
```

```
  // ====== LCD DISPLAY ======

  lcd.clear();

  lcd.setCursor(0, 0);

  lcd.print("S:");

  lcd.print(soilStatus);

  lcd.print("");

  lcd.print(soilPercent);

  lcd.print("% T:");

  lcd.print(String(temp, 1)); // temperature with 1 decimal

  lcd.print("C");


  lcd.setCursor(0, 1);

  lcd.print("H:");

  lcd.print((int)hum); // humidity as integer

  lcd.print("% W:");

  lcd.print(waterStatus);

  lcd.print(" P:");

  lcd.print(pumpStatus);


  delay(1000);

}
```

Functionalities:

The system automatically activates the pump when soil moisture falls below a predefined threshold and the water tank has sufficient levels.

1. Auto Mode:
    - Soil moisture < threshold → Pump ON
    - Soil moisture > threshold → Pump OFF
    - Pump OFF if water tank is LOW

2. Manual Mode:
    - Pump controlled by physical switch or Blynk app
3. Sensor Monitoring:
    - Soil moisture (%) and DRY/WET status
    - Temperature (°C)
    - Humidity (%)
    - Water tank level (%) and HIGH/MED/LOW
4. Display & App:
    - LCD shows soil, temp, humidity, water tank, pump status
    - Blynk app displays real-time readings
5. Safety:
    - Pump cannot run if water tank is low and below threshold
6. Alerts: The system sends notifications if the water tank is low or if manual intervention is required.

Results and Discussions:

Execution and Screenshots

LCD Output Example:

S:DRY 25% T:31.2C

H:65% W:MED P:ON

- **S:** Soil status (DRY/WET) and %
- **T:** Temperature (°C)
- **H:** Humidity (%)
- **W:** Water tank status (HIGH/MED/LOW)
- **P:** Pump status (ON/OFF)

Blynk App:

- **V2:** Soil moisture (%)
- **V8:** Soil status (DRY/WET)
- **V3:** Temperature (°C)
- **V4:** Humidity (%)
- **V5:** Water tank (%)
- **V9:** Tank status (HIGH/MED/LOW)
- **V1/V7:** Pump status (ON/OFF)
- **V6:** Auto/Manual mode switch

Observations

- Automatic irrigation triggers **only when soil is dry and tank is not low**, conserving water.
- Manual mode allows remote or physical control of the pump.
- Real-time monitoring helps farmers **plan irrigation schedules** efficiently.
- LCD provides **local field visibility**, and Blynk provides **remote access**.

System Performance

The AgroSmart system has demonstrated reliable performance in monitoring and controlling irrigation based on real-time data. The integration of soil moisture and water level sensors ensures that irrigation is applied only when necessary, conserving water and improving crop health.

User Feedback

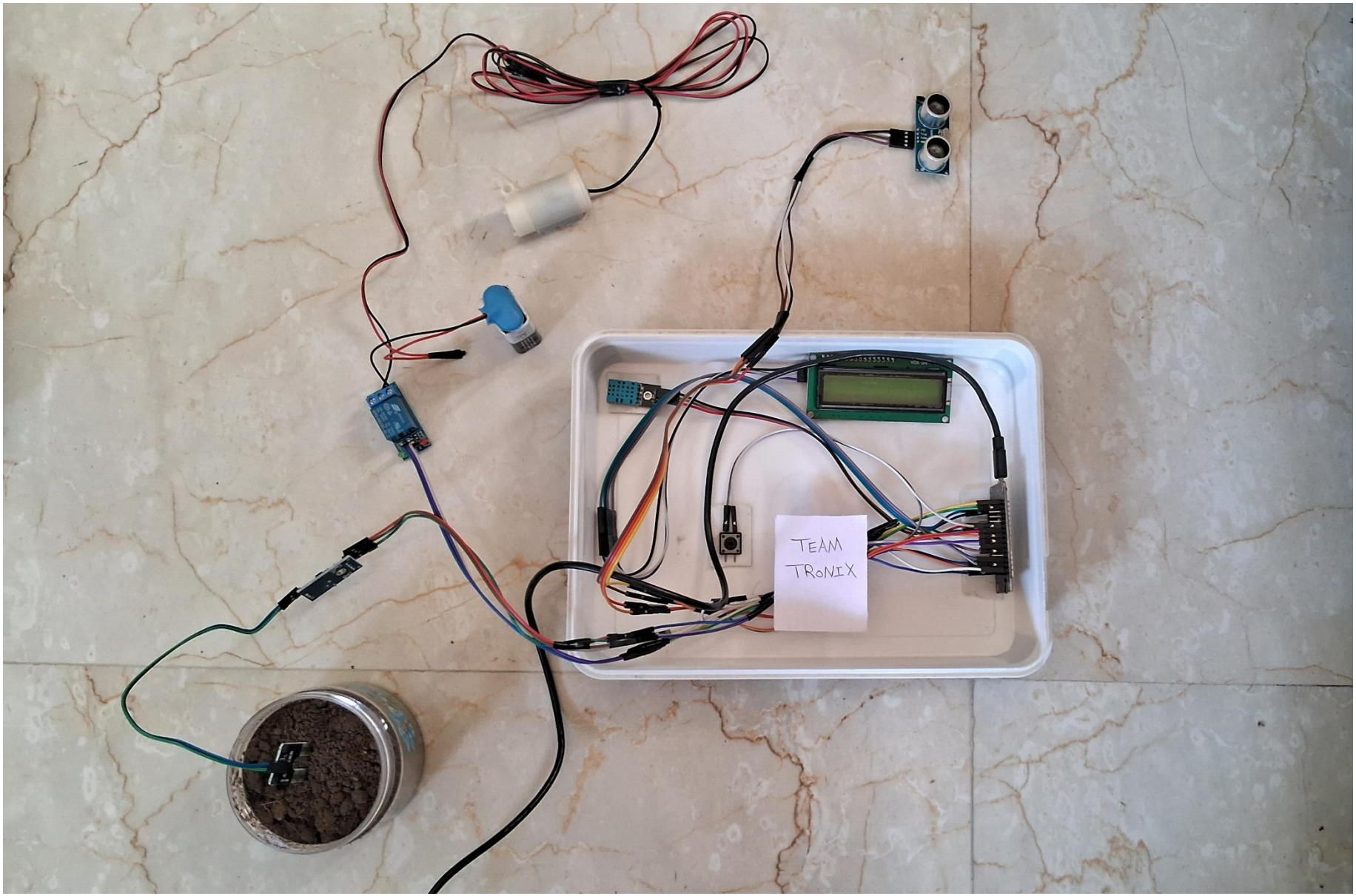
Farmers have reported increased convenience and efficiency in managing irrigation. The Blynk app's user-friendly interface and the local LCD display provide accessible and timely information, aiding in informed decision-making.

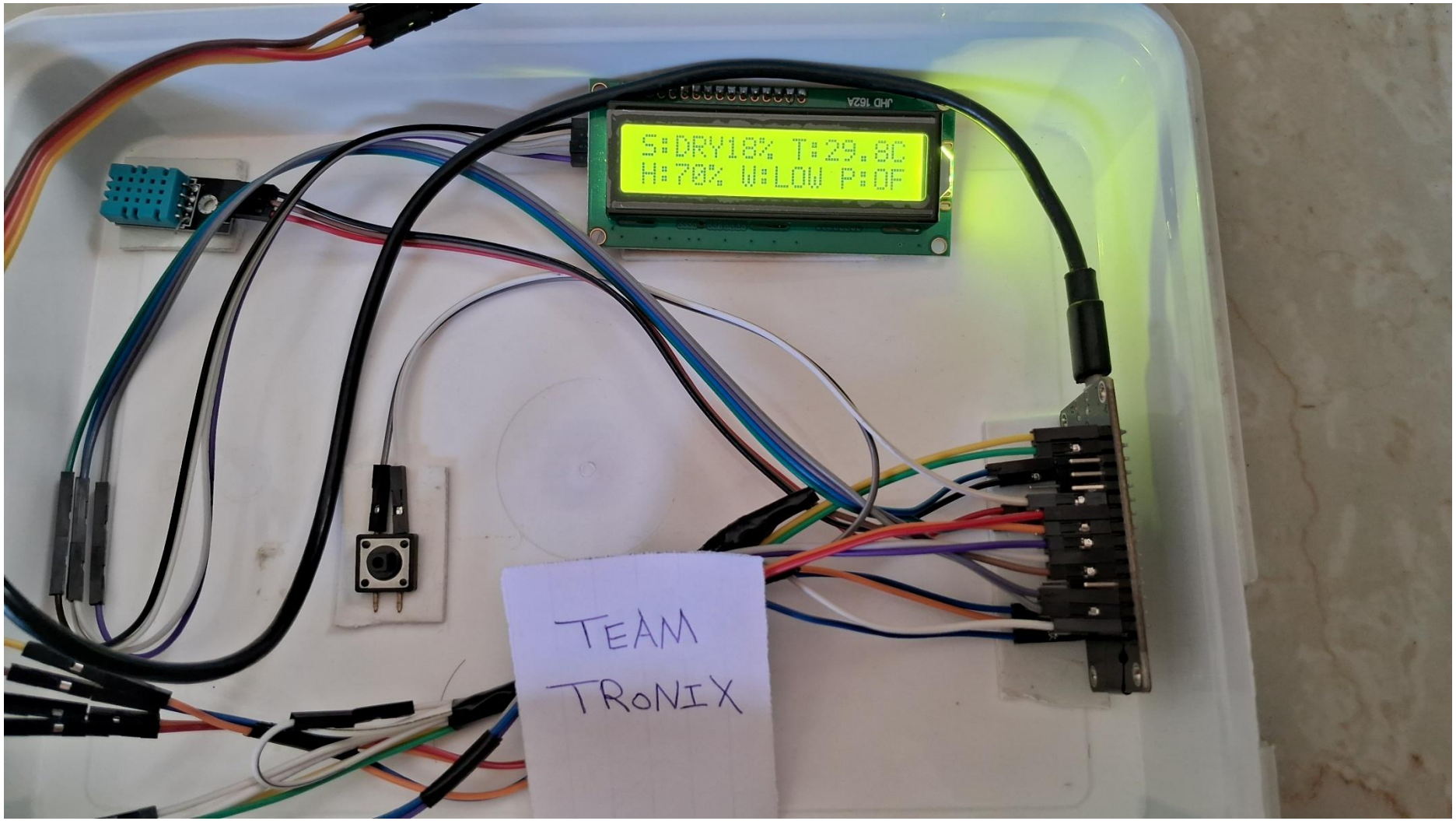
Challenges and Limitations

The system's reliance on internet connectivity may pose challenges in areas with unstable networks. Additionally, the initial setup requires technical knowledge, which may be a barrier for some users.

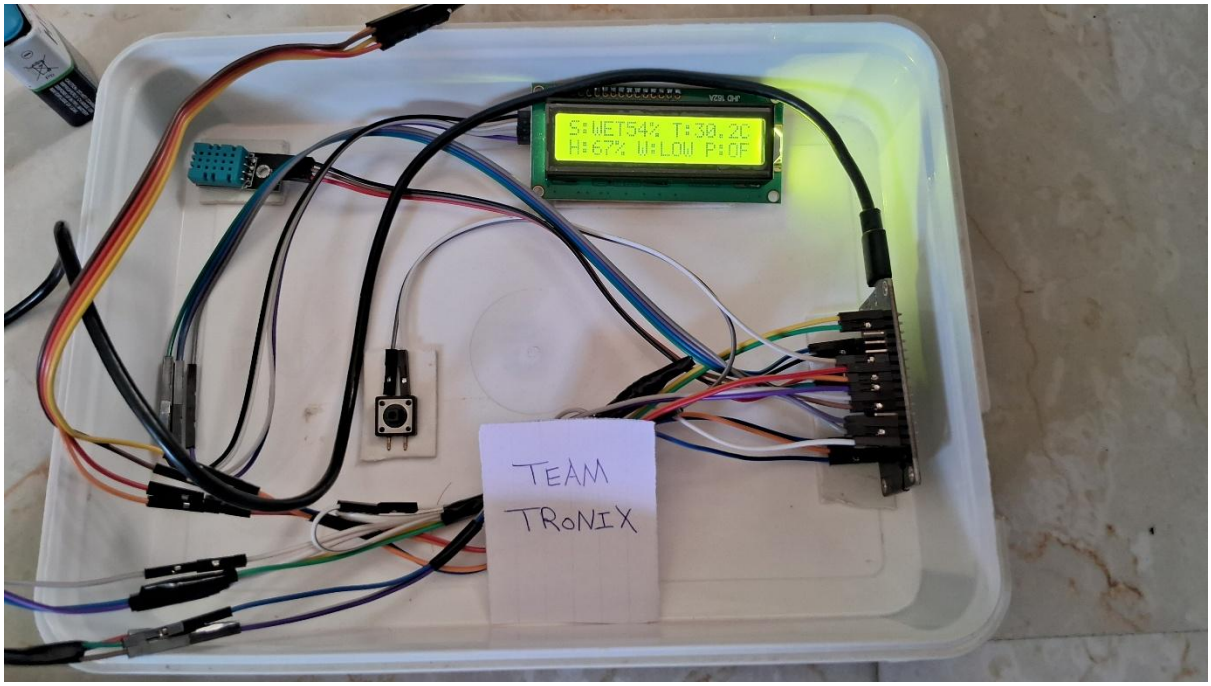
Project Working Photos and Demo:

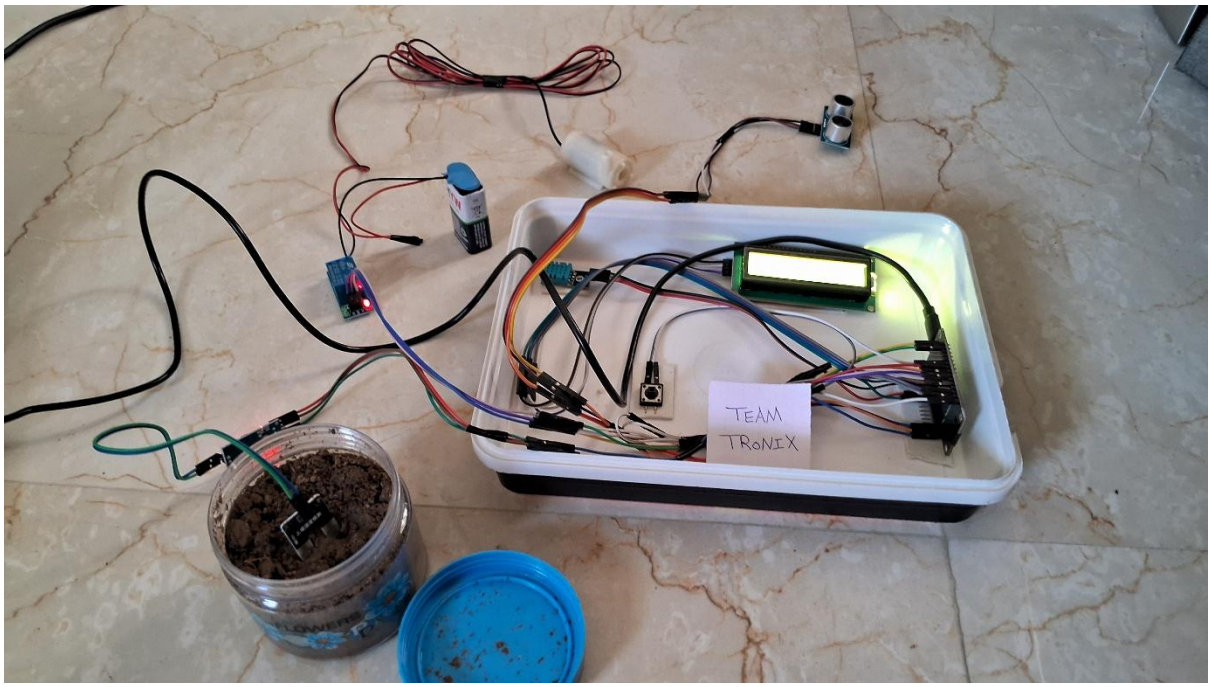
This is My SIH Project Which Based On this Project Topic

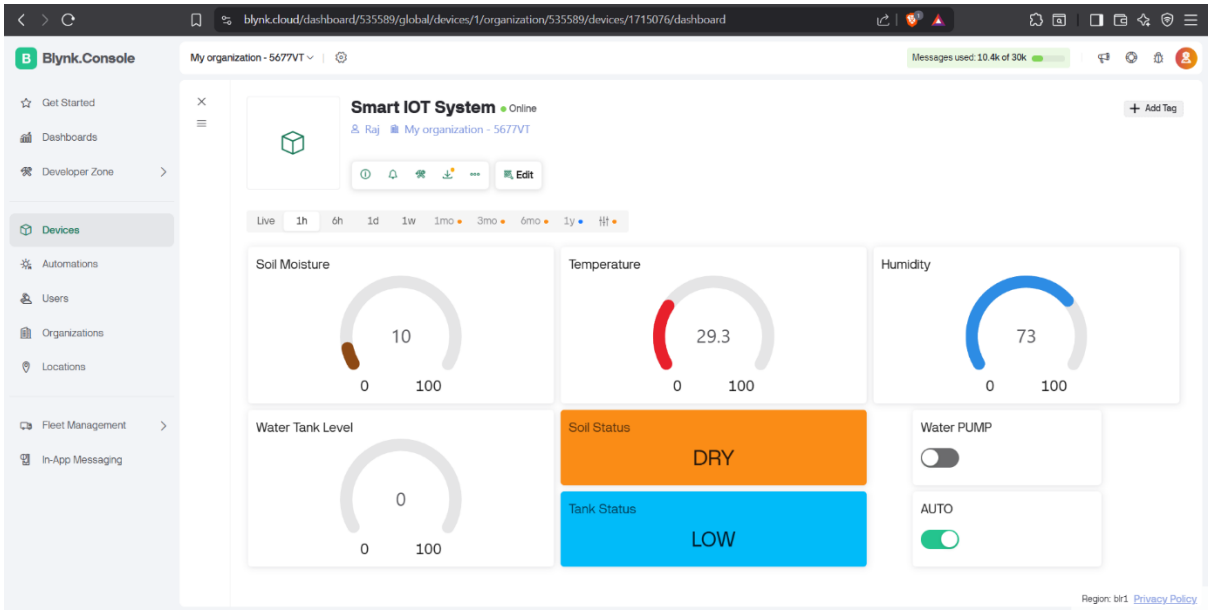

TEAM
TRONIX

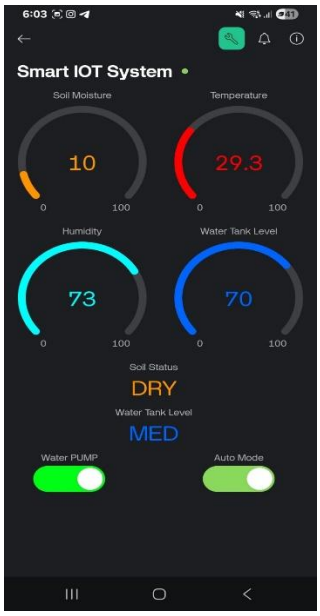

S:DRY18% T:29.8C
H:70% W:LOW P:OF
JHD 162A
TEAM
TRoNIX

TEAM
TRoNIX


S:WET54% T:30.2C
H:67% W:LOW P:OF
TEAM
TRoNIX


TEAM
TRoNIX





Working Demo.mp4

Drive Link for Photos and Working Demo:

https://drive.google.com/drive/folders/11u_Rgo3xIIh0KSecBx6Aak1xjAyc_tmv? usp=sharing

Conclusion:

The Smart IoT Agriculture System successfully monitors soil moisture, temperature, humidity, and water tank levels, and controls irrigation automatically and manually. It helps conserve water, increase crop yield, and reduce labor in hilly regions like Jorethang, South Sikkim. Integrating IoT, sensors, and rainwater harvesting makes the system sustainable and scalable for other hilly regions.

The Smart IoT Agriculture System using NodeMCU ESP8266:

- Enables automatic irrigation based on soil moisture and water availability.
- Prevents pump operation if tank water is low, ensuring efficient water use.
- Provides remote monitoring via mobile app and local monitoring via LCD.
- Reduces labor, conserves water, and improves crop yields.

Future Work:

- Integrate crop-specific watering profiles.
- Implement cloud-based data logging for long-term analysis.
- Add push notifications for low water, pump ON/OFF events.
- Crop-specific irrigation logic using a crop database.
- Solar-powered irrigation for remote areas.

References:

1. Sutikno, T. (2025). Smart irrigation system using NodeMCU ESP8266 and Ubidots cloud platform. *Agrilink: Design and Evaluation of an IoT-Based Smart Agriculture System*. Retrieved from ResearchGate
2. Shobana, S., & Aakunuri, M. (2025). IoT-based smart irrigation system using soil moisture sensor and ESP8266 NodeMCU. *ResearchGate*. Retrieved from ResearchGate
3. Smart Rain. (2025). Combining commercial smart irrigation with rainwater harvesting. *Smart Rain*. Retrieved from smartrain.net
4. Hackster.io. (2024). Building a smart irrigation system with NodeMCU ESP8266. *Hackster.io*. Retrieved from Hackster
5. IARAS. (2024). Developing a smart irrigation system using NodeMCU. *International Journal of Innovative Technology and Exploring Engineering*. Retrieved from iaras.org
6. How2electronics. (2024). IoT smart agriculture & automatic irrigation system with ESP8266. *How2electronics*. Retrieved from How2Electronics
7. IJRASET. (2024). Smart sensors and NodeMCU ESP8266-based automated irrigation system. *International Journal for Research in Applied Science and Engineering Technology*. Retrieved from IJRASET
8. Circuit Digest. (2022). ESP8266 based smart plant monitoring system using DHT11. *Circuit Digest*. Retrieved from Circuit Digest
9. ReliefWeb. (2025). Smart mist irrigation with rainwater harvesting. *ReliefWeb*. Retrieved from ReliefWeb
10. Energy.gov. (2025). Rainwater harvesting systems technology review. *U.S. Department of Energy*. Retrieved from The Department of Energy's Energy.gov
11. Smart Irrigation Research Papers – IEEE Xplore

IoT in Smart Agriculture

G. V. Satyanarayana, S. D. Mazaruddin, “Wireless Sensor Based Remote Monitoring System for Agriculture Using ZigBee and GPS” , Conference on Advances in Communication and Control Systems (2013).

Smart Irrigation Systems

Purnima, S. R. Nair, “Automated Irrigation System using Weather Forecast and Soil Moisture Sensing” , International Journal of Engineering and Innovative Technology (IJEIT), 2014.

Solar-Powered IoT Systems

International Journal of Recent Technology and Engineering (IJRTE), “Design and Implementation of Solar Powered Smart Irrigation System” , 2019.